LABORATORIO 06

Sensor I2C: Medición de Temperatura y Humedad con BMP280 / DHT

Objetivo:

Implementar la comunicación mediante el protocolo I2C entre un microcontrolador y un sensor ambiental. Además, identificar dispositivos conectados al bus I2C, obtener datos de manera periódica y enviarlos correctamente formateados a través de comunicación UART.

Materiales:

- Arduino UNO, Nano o ESP32 DevKit

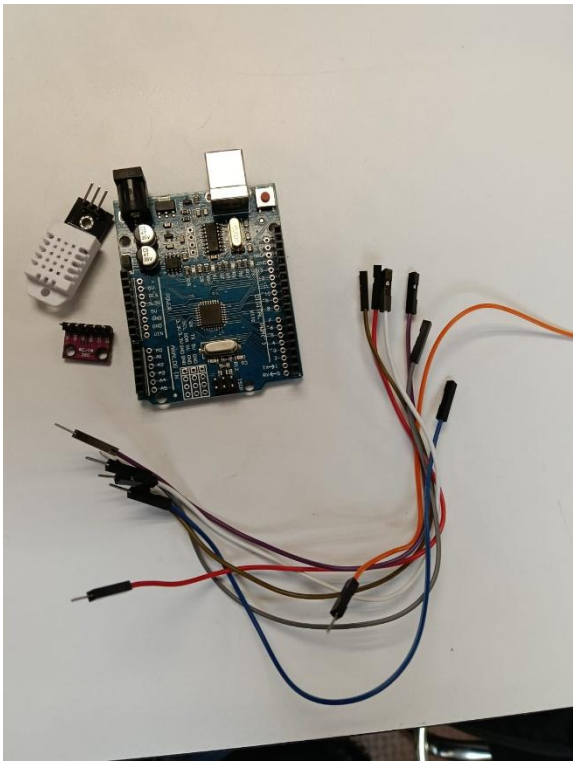
- Sensor BMP280 (también puede ser BMP180 o BME280) con módulo que incluya resistencias pull-up

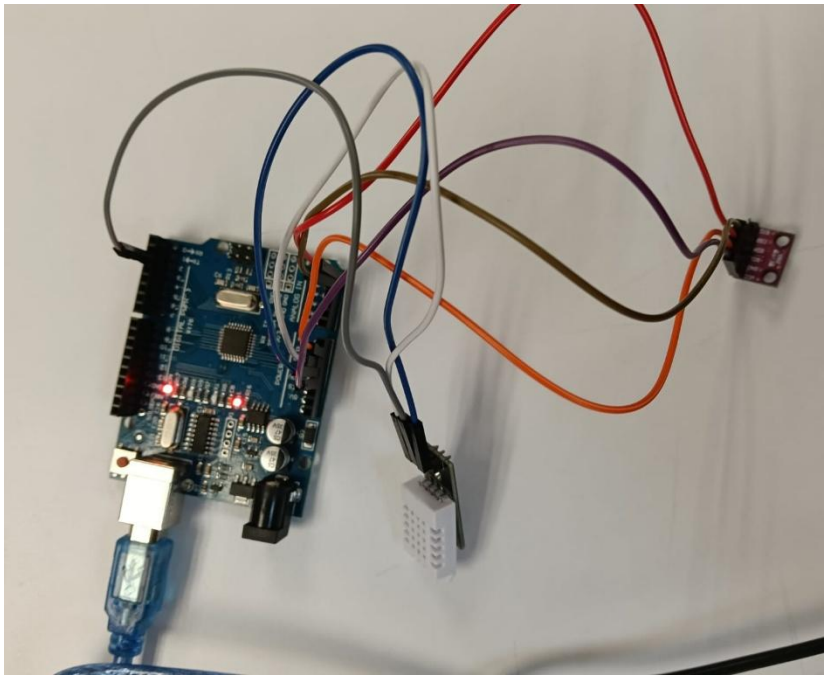
- Opcional: sensor DHT22 con resistencia pull-up de 10 kΩ

- Resistencias pull-up de 4.7 kΩ (en caso de que el módulo no las tenga integradas)

- Protoboard y cables jumper (macho-macho y macho-hembra)

- Arduino IDE versión 2.x con la librería “Adafruit BMP280 Library” instalada





Sensor I2C > src > main.cpp > setup()

```cpp
#include <Arduino.h>

// —— Lectura BMP280 con reporte estructurado ——
#include <Wire.h>
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp;
const float PRESION_MAR = 1013.25;  // hPa al nivel del mar
unsigned long ultimaLectura = 0;

const long INTERVALO = 3000;  // leer cada 3 segundos
int numerolectura = 0;

void setup() {
    Serial.begin(115200);
    Serial.println("Lab 06 - Sensor BMP280 via I2C");
    if (!bmp.begin(0x76)) {  // probar 0x77 si falla
        Serial.println("ERROR: BMP280 no encontrado. Verificar conexion y direccion I2C.");
        while (1) delay(100);  // detener ejecucion
    }
    // Configuracion: modo normal, oversampling x2
    bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                    Adafruit_BMP280::SAMPLING_X2,
                    Adafruit_BMP280::SAMPLING_X16,
                    Adafruit_BMP280::FILTER_X16,
```

```
Executing task in folder Sensor I2C: C:\Users\herna\.platformio\penv\Scripts\platformio.exe device monitor
342 | 30.2 C   | 1004.99 hPa | 69.0 m
343 | 30.3 C   | 1005.00 hPa | 68.9 m
344 | 30.3 C   | 1005.01 hPa | 68.9 m
345 | 30.4 C   | 1005.01 hPa | 68.9 m
346 | 30.4 C   | 1005.00 hPa | 68.9 m
347 | 30.5 C   | 1005.00 hPa | 68.9 m
348 | 30.5 C   | 1004.99 hPa | 69.0 m
349 | 30.5 C   | 1004.98 hPa | 69.1 m
350 | 30.5 C   | 1004.98 hPa | 69.1 m
351 | 30.6 C   | 1004.97 hPa | 69.1 m
```

Sensor I2C > src > main.cpp > setup()

```cpp
#include <Arduino.h>

// —— Lectura BMP280 con reporte estructurado ——
#include <Wire.h>
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp;
const float PRESION_MAR = 1013.25;  // hPa al nivel del mar
unsigned long ultimaLectura = 0;

const long INTERVALO = 3000;  // leer cada 3 segundos
int numerolectura = 0;

void setup() {
    Serial.begin(115200);
    Serial.println("Lab 06 - Sensor BMP280 via I2C");
    if (!bmp.begin(0x76)) {  // probar 0x77 si falla
        Serial.println("ERROR: BMP280 no encontrado. Verificar conexion y direccion I2C.");
        while (1) delay(100);  // detener ejecucion
    }
    // Configuracion: modo normal, oversampling x2
    bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                    Adafruit_BMP280::SAMPLING_X2,
                    Adafruit_BMP280::SAMPLING_X16,
                    Adafruit_BMP280::FILTER_X16,
```

```
Executing task in folder Sensor I2C: C:\Users\herna\.platformio\penv\Scripts\platformio.exe device monitor
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
Lab 06 - Sensor BMP280 via I2C
BMP280 inicializado correctamente.
N | Temp(C) | Presion(hPa) | Altitud(m)
--|--------|------------|---------
1 | 24.4 C   | 1005.46 hPa | 65.0 m
2 | 24.4 C   | 1005.47 hPa | 65.0 m
3 | 24.4 C   | 1005.47 hPa | 65.0 m
4 | 24.4 C   | 1005.46 hPa | 65.0 m
5 | 24.3 C   | 1005.47 hPa | 65.0 m
```

PREGUNTAS DE ANÁLISIS Y REPORTE

¿Cuál fue la dirección I2C detectada para el sensor? ¿Qué determina si es 0x76 o 0x77?

El sensor puede aparecer en la dirección 0x76 o 0x77 dependiendo del estado del pin SDO. Si este pin está conectado a tierra (GND), se asigna una dirección; si está conectado a voltaje (VCC), cambia a la otra.

¿Cuántos dispositivos pueden conectarse en un mismo bus I2C? ¿Qué ocurre si dos tienen la misma dirección?

En teoría, se pueden conectar hasta 127 dispositivos en el mismo bus. Sin embargo, si dos dispositivos comparten la misma dirección fija, el microcontrolador no podrá diferenciarlos, generando un conflicto de comunicación.

¿Por qué se utiliza millis() en lugar de delay(3000) para controlar el tiempo entre lecturas? Da un ejemplo donde delay() sería problemático.

La función millis() permite medir el tiempo sin detener la ejecución del programa, mientras que delay() pausa completamente el microcontrolador. Por ejemplo, si se necesita leer sensores y al mismo tiempo responder a comandos seriales, usar delay() impediría esa multitarea.

¿La altitud obtenida por el sensor coincide con la real? ¿Por qué puede haber diferencias?

La altitud no se mide directamente, sino que se calcula a partir de la presión atmosférica. Debido a que la presión también varía por condiciones climáticas como temperatura, lluvia o cambios de clima, el valor calculado puede no coincidir exactamente con la altitud real.

Si deseas agregar una pantalla OLED SSD1306 (dirección 0x3C) al mismo bus I2C, ¿es necesario modificar las conexiones SDA y SCL?

No es necesario cambiar el cableado, ya que todos los dispositivos I2C comparten las mismas líneas SDA y SCL. Lo importante es que cada dispositivo tenga una dirección distinta para evitar conflictos.